

MET CS688

WEB ANALYTICS AND MINING

ZLATKO VASILKOSKI

FLASK AND SWAGGER UI

What is API

- API (Application Programming Interface) is a way to communicate with an application.
- REST (REpresentational State Transfer) is an architectural style that aims
  - For stateless communications and
  - Separates client and server concerns.
- A REST API is a type of web service that uses HTTP requests to GET, PUT, POST and DELETE data.
- It is a way for two computer systems to communicate over the internet or other network, usually in the form of JSON or XML data.
- The most popular type of REST API is a RESTful API, which uses HTTP verbs to for the desired action to be performed.
  - For example, a GET request to a RESTful API will retrieve data, while a POST request will create a new entry.
- The REST API provides a stateless, client-server architecture, allowing clients to **access** resources without having to **store** state information on the server.
- An **endpoint** is a specific URL or IP address of a server that provides access to the resources of a REST API.
  - It is the point where the client (such as a web browser) makes a request to the server.
  - Endpoints can be used to perform various operations, such as creating, reading, updating, and deleting data.

What is Python's Flask

- Python **Flask** is a micro web framework written in Python.
- It provides a simple web application development framework for Python and allows running web applications.
  - Several other frameworks available for Python are Tornado, Pyramind, and Django.
  - Flask is one of the most used web development frameworks.
- Flask is designed to make it easier to build web applications by providing a simple and powerful routing system, support for multiple templating languages, and a flexible database layer.
- It also provides support for extensions, allowing developers to add functionality to their applications quickly and easily.
- The minimal code for creating a Flask app is :

```python
# main.py
from flask import Flask
app = Flask(__name__)

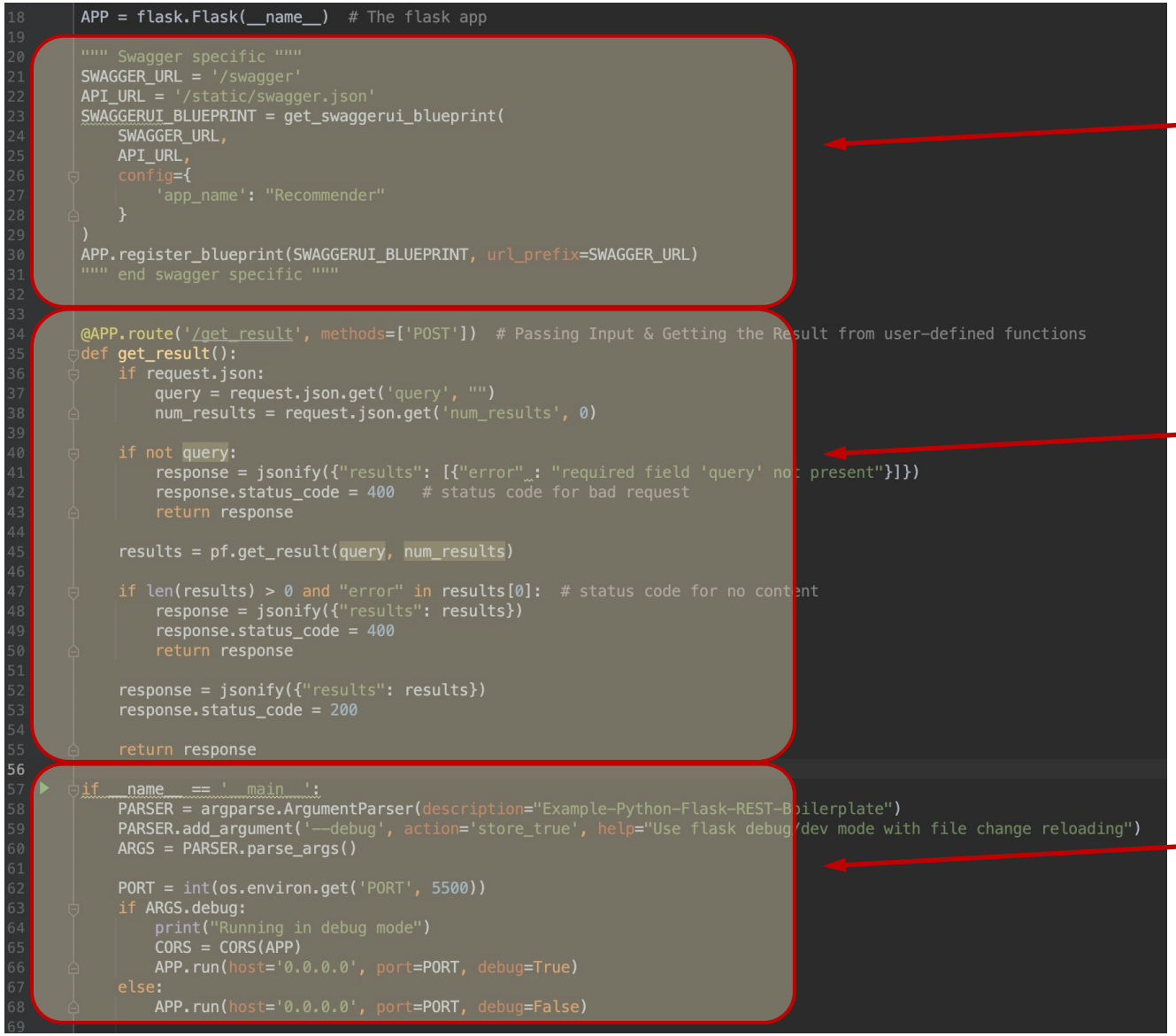
@app.route('/basic_api/hello_world')
def hello_world():
    return 'Hello, World!'
```

Swagger UI

- **Swagger UI** is a web-based user interface that allows developers to interact with web-based APIs.
- It provides an interactive documentation system that allows users to explore and test APIs without writing any code.
- The Swagger UI also allows developers to generate client code in **multiple languages** and frameworks, making it easier to quickly build projects.
- It also provides an easy-to-use graphical interface for developers to explore their APIs.
- Using Flask and Swagger UI in Python, developers can quickly and easily create APIs.
  - First, creating a Flask application is needed and
    - Configured routes and endpoints will be used to access the API.
  - Once the routes and endpoints are configured, then Swagger UI can be created with documentation for the API.
  - This will allow users to explore the API and test it out.
  - Finally, developers can use Swagger UI to generate client code in multiple languages and frameworks, making it easier to quickly build projects.

Swagger UI Example

- After in “app.py” Flask is instantiated, we need to add some Swagger specific blueprint code.



```
APP = flask.Flask(__name__)  # The flask app

""" Swagger specific """
SWAGGER_URL = '/swagger'
API_URL = '/static/swagger.json'
SWAGGERUI_BLUEPRINT = get_swaggerui_blueprint(
    SWAGGER_URL,
    API_URL,
    config={
        'app_name': "Recommender"
    }
)
APP.register_blueprint(SWAGGERUI_BLUEPRINT, url_prefix=SWAGGER_URL)
""" end swagger specific """

@APP.route('/get_result', methods=['POST'])  # Passing Input & Getting the Result from user-defined functions
def get_result():
    if request.json:
        query = request.json.get('query', "")
        num_results = request.json.get('num_results', 0)

    if not query:
        response = jsonify({"results": [{"error": "required field 'query' not present"}]})
        response.status_code = 400   # status code for bad request
        return response

    results = pf.get_result(query, num_results)

    if len(results) > 0 and "error" in results[0]:  # status code for no content
        response = jsonify({"results": results})
        response.status_code = 400
        return response

    response = jsonify({"results": results})
    response.status_code = 200

    return response

if __name__ == '__main__':
    PARSER = argparse.ArgumentParser(description="Example-Python-Flask-REST-Boilerplate")
    PARSER.add_argument('--debug', action='store_true', help="Use flask debug/dev mode with file change reloading")
    ARGS = PARSER.parse_args()

    PORT = int(os.environ.get('PORT', 5500))
    if ARGS.debug:
        print("Running in debug mode")
        CORS = CORS(APP)
        APP.run(host='0.0.0.0', port=PORT, debug=True)
    else:
        APP.run(host='0.0.0.0', port=PORT, debug=False)
```

Swagger specific blueprint code that also references “/static/swagger.json” in which the UI components and schemas are defined.

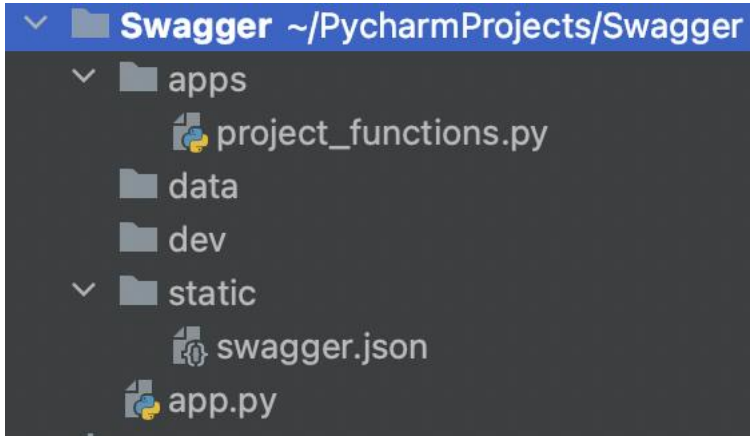
Request method that calls the function “get_result()” defined in “apps/project_functions.py” andin addition to Error handling, processes the response.

App code

Swagger UI Example

- The following example creates a URI (Uniform Resource Identifier) at the /swagger endpoint, and it returns a file called /swagger.json which will be parsed inside a self hosted Swagger-UI front end.
- The folder “static” contains the file “swagger.json” that specifies the UI components and schemas. It looks like this:


```
{
  "openapi": "3.0.0",
  "info": {
      "description": "Swagger UI for calling App endpoint.",
      "version": "1.0.0",
      "title": "Swagger Example App"
  },
  "paths": {
    "/get_result": {
      "post": {
        "tags": [
          "Swagger Example Request"
        ],
        "summary": "Get Request",
        "requestBody": {
          "description": "Input Post Object",
          "required": true,
          "content": {
            "application/json": {
              "schema": {
                "$ref": "#/components/schemas/appRequestPostBody"
              },
              "example": {
                  "query": "Is there any investment guide for the stock market in India?",
                  "num_results": 2
              }
            }
          }
        },
        "produces": [
          "application/json"
        ],
        "responses": {
          "201": {
            "description": "OK",
            "schema": {
              "$ref": "#/components/schemas/appResponseBody"
            }
          },
          "400": {
            "description": "Failed. Bad post data."
          }
        }
      }
```

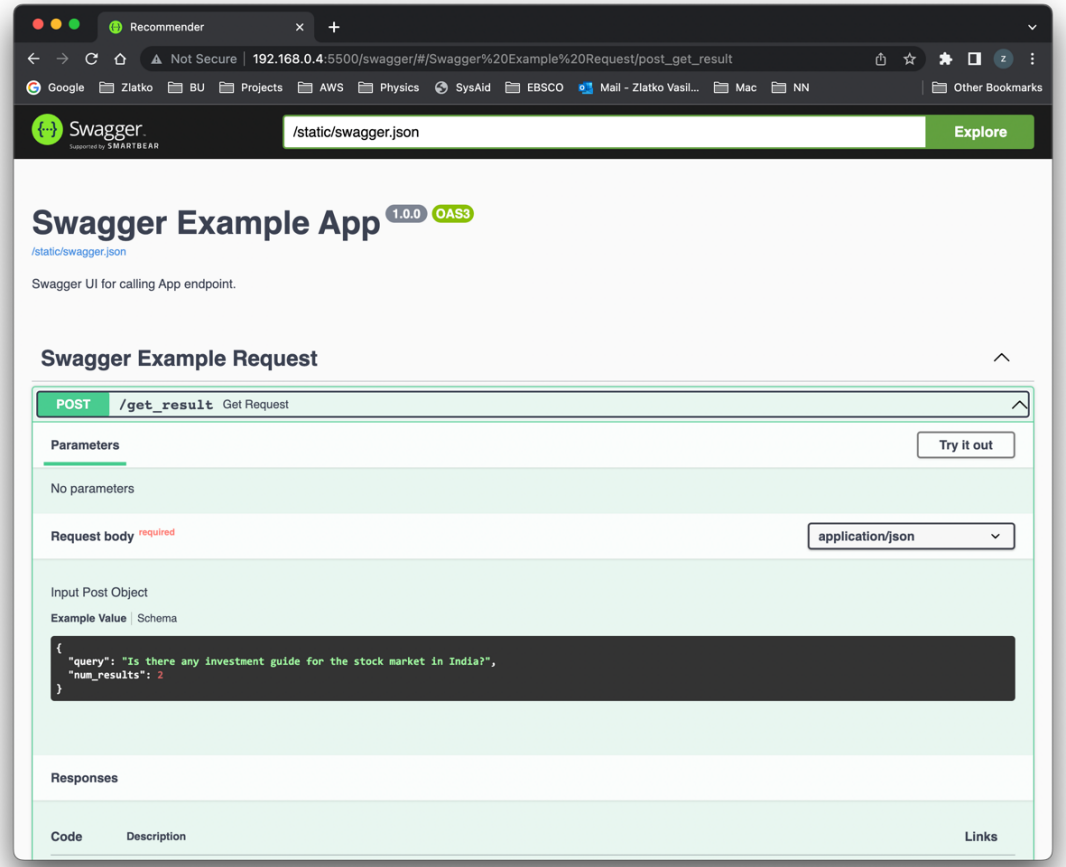



Swagger UI Example

- After running the file “app.py” in PyCharm

```
Swagger
/usr/local/anaconda3/envs/flaskProject/bin/python /Users/zvasilkoski/PycharmProjects/Swagger/app.py
 * Serving Flask app 'app'
 * Debug mode: off
WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
 * Running on all addresses (0.0.0.0)
 * Running on http://127.0.0.1:5500
 * Running on http://192.168.0.4:5500
Press CTRL+C to quit
```

- Open a browser and type the following URL: http://127.0.0.1:5500/swagger/#/**Swagger**
- Note:
  - The port “5500” is specified in “app.py”
  - The word “swagger” in URL is needed at the end
- The interactive output looks something like this:



Swagger UI Example

- Click on the button “Try it out”
- Then you can interactively change the content of the input Post object which is

```
{
  "query": "Is there any investment guide for the stock market in India?",
  "num_results": 2
}
```

- By clicking “Execute” you pass the input (the “query” and the “num_results”) to “get_results()” which simply loops and generates a list of “num_results” responses, each containing the [query, index, score]

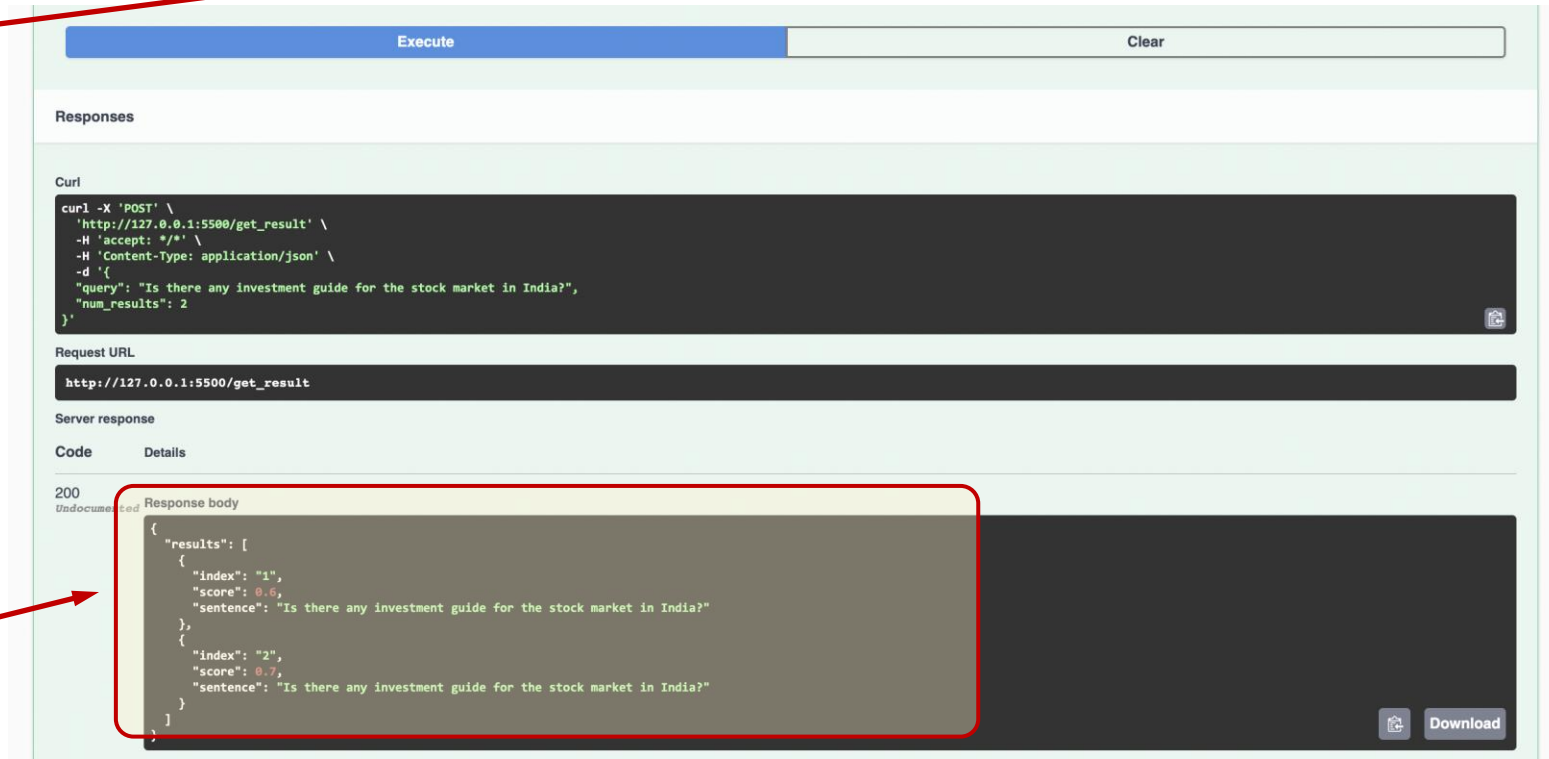
```
"""
    Functions Used in this project
"""


def get_result(query, n):

    data = []
    index, score = 0, 0.5

    for records in range(n):
        index += 1
        score += 0.1
        item = {"sentence": query, "index": str(index), "score": score}
        data.append(item)

    return data
```



The Result (the API response)

A

- A

Streamlit Dashboard

- Another UI is Streamlit
- It turns data scripts into shareable web apps, as easy as writing Python scripts.
- Create beautiful apps for your ML code without needing another complicated to learn tool.
- The minimal code for creating a Streamlit app is :
- Streamlit has widgets that are treated as variables.

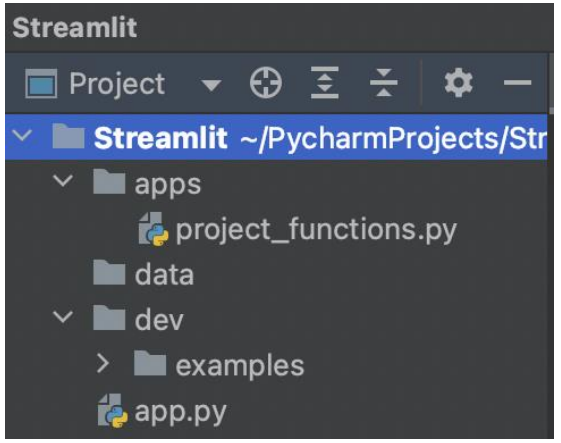
```
import streamlit as st

st.write('Hello, world!')
```

- Here is a link to the Streamlit Galery: https://streamlit.io/gallery

Streamlit Dashboard

- Here is the previous Swagger App rewritten in just few lines using Streamlit.




```python
import pandas as pd
import streamlit as st
import apps.project_functions as pf


@st.cache
def load_data(path):  # Downloads file & caches it in memory
    dataset = pd.read_csv(path)
    return dataset


with st.sidebar:
    st.subheader('About the Dashboard')
    st.markdown(' Stremlit UI for calling App endpoint.')
    st.header('Example Request')
    query = st.text_input('Query', 'Is there any investment guide for the stock market in India?')
    num_results = st.text_input('Number of Results Returned', 2)
    num_results = int(num_results)
    submitted = st.button("Submitt")


st.title('Streamlit App')  # Or st.write('# First Streamlit App')
st.markdown("""
    Stremlit Example Request Dashboard
    """)


st.header('Returned Response')

if submitted:
    results = pf.get_result(query, num_results)
    st.write(results)
```

Cache loaded data in memory, if needed for faster response.

Defining the sidebar and the request.

Defining the Panel title.

Defining the Response.

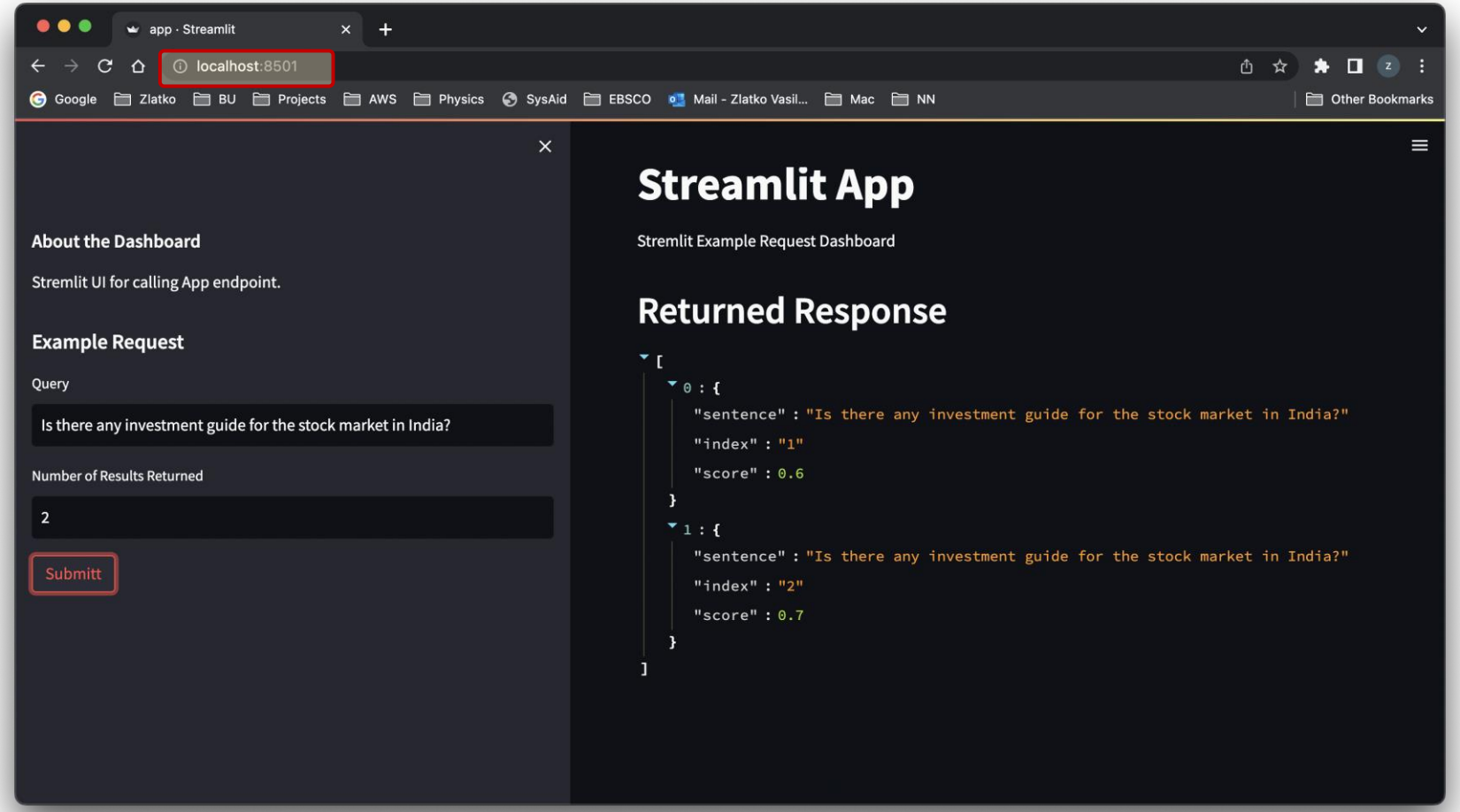
Streamlit Dashboard

- You run Streamlit “app.py” from a terminal.
  - streamlit run app.py
- Here is the output that appears in a browser:

```
(Streamlit) zvasilkoski@x86_64-apple-darwin13 Streamlit % streamlit run app.py

  You can now view your Streamlit app in your browser.

  Local URL: http://localhost:8501
  Network URL: http://192.168.0.4:8501
```



A

- A

A

- A